SchoolofComputerScienceandArtificial Intelligence

LabAssignment#20.2

Program :B.Tech(CSE)

Specialization :AIML

Course Title :AIAssistedCoding

Course Code : 23CS002PC304

Semester : VI

AcademicSession:2025-2026

Name of Student :P.Sushanthi

Enrollment No. : 2303A52102

Batch No. : 33

Date :11/04/26

Task1–PasswordSecurityEnhancementPrompt:

Generate a Python user registration system storing username and password.

Code:

Python User Registration System

This system provides a `UserAuth` class with methods to register new users and authenticate existing ones. For simplicity, credentials are stored in a dictionary in memory. In a real-world application, you would use a database and secure password hashing.

```
class UserAuth:
    def __init__(self):
        self.users = {}

    def register(self, username, password):
        if username in self.users:
            print(f"User '{username}' already exists.")
            return False
        self.users[username] = password # In a real system, hash the password!
        print(f"User '{username}' registered successfully.")
        return True

    def login(self, username, password):
        if username not in self.users:
            print(f"User '{username}' not found.")
            return False
        if self.users[username] == password:
            print(f"User '{username}' logged in successfully.")
            return True
        else:
            print("Incorrect password.")
            return False

    def get_users(self):
        return list(self.users.keys())
```

Demonstration of the User Registration System

```
# Create an instance of the UserAuth system
auth_system = UserAuth()

# Register new users
auth_system.register("alice", "password123")
auth_system.register("bob", "securepass")
auth_system.register("alice", "anotherpass") # Attempt to register existing user

print("\n--- Current Users ---")
print(auth_system.get_users())

# Test login functionality
print("\n--- Login Attempts ---")
auth_system.login("alice", "password123") # Successful login
auth_system.login("bob", "wrongpass")     # Incorrect password
auth_system.login("charlie", "newuser")   # User not found
auth_system.login("bob", "securepass")    # Successful login
```

```
User 'alice' registered successfully.
User 'bob' registered successfully.
User 'alice' already exists.

--- Current Users ---
['alice', 'bob']

--- Login Attempts ---
User 'alice' logged in successfully.
Incorrect password.
User 'charlie' not found.
User 'bob' logged in successfully.
True
```

Task2–

SecureAPIKeyManagementPrompt:

Generate Python code to call an API using API key

Code:

Now, let's make a GET request to a public API. We'll use the JSONPlaceholder API for demonstration purposes, which provides dummy data without requiring authentication. For example, we can fetch a list of posts.

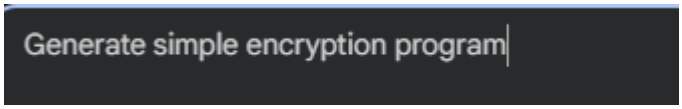
```
import requests

# Define the API endpoint URL
api_url = "https://jsonplaceholder.typicode.com/posts"

try:
    # Make a GET request to the API
    response = requests.get(api_url)

    # Check if the request was successful (status code 200)
    if response.status_code == 200:
        # Parse the JSON response
        data = response.json()

        print("API call successful!")
        print(f"Number of posts retrieved: {len(data)}")
        print("First 3 posts:")
        # Display the first 3 items for brevity
        for i, post in enumerate(data[:3]):
            print(f"  Post {i+1}: {post['title'][:50]}...")

    else:
        print(f"API call failed with status code: {response.status_code}")
        print(f"Response content: {response.text}")

except requests.exceptions.RequestException as e:
    print(f"An error occurred during the API call: {e}")
```

```
API call successful!
Number of posts retrieved: 100
First 3 posts:
  Post 1: sunt aut facere repellat provident occaecati excep...
  Post 2: qui est esse...
  Post 3: ea molestias quasi exercitationem repellat qui ips...
```

Explanation:

- `import requests` : Imports the `requests` library, which simplifies making HTTP requests.
- `api_url = "https://jsonplaceholder.typicode.com/posts"` : Defines the URL of the API endpoint you want to call.
- `requests.get(api_url)` : Sends a GET request to the specified URL. For APIs that don't require an API key, this is often all you need.
- `response.status_code == 200` : Checks if the request was successful. A status code of 200 indicates success.
- `response.json()` : Parses the JSON response body into a Python dictionary or list.
- `try...except requests.exceptions.RequestException` : This block handles potential errors during the network request, such as network issues or invalid URLs.

Task3–WeakEncryptionDetectionPrompt:



Code:

Simple Encryption Program: Caesar Cipher

A Caesar cipher is one of the simplest and most widely known encryption techniques. It's a type of substitution cipher in which each letter in the plaintext is replaced by a letter some fixed number of positions down or up the alphabet. For example, with a left shift of 3, D would be replaced by A, E would become B, and so on.

```python
def caesar_cipher(text, shift, mode='encrypt'):
    '''
    Encrypts or decrypts a message using the Caesar cipher.

    Args:
        text (str): The input string to encrypt or decrypt.
        shift (int): The number of positions to shift each letter.
        mode (str): 'encrypt' for encryption, 'decrypt' for decryption.

    Returns:
        str: The encrypted or decrypted string.
    '''
    result = ""

    if mode == 'decrypt':
        shift = -shift # Reverse the shift for decryption

    for char in text:
        if char.isalpha(): # Check if the character is an alphabet letter
            start = ord('a') if char.islower() else ord('A')
            # Apply the shift and wrap around the alphabet (26 letters)
            shifted_char = chr(start + (ord(char) - start + shift) % 26)
            result += shifted_char
        else:
            result += char # Keep non-alphabetic characters as they are
    return result
```

Demonstration of the Caesar Cipher

```python
# Define the message and the shift value
message = "Hello, World!"
shift_value = 3

# Encrypt the message
encrypted_message = caesar_cipher(message, shift_value, mode='encrypt')
print(f"Original Message: {message}")
print(f"Shift Value: {shift_value}")
print(f"Encrypted Message: {encrypted_message}")

# Decrypt the message
decrypted_message = caesar_cipher(encrypted_message, shift_value, mode='decrypt')
print(f"Decrypted Message: {decrypted_message}")
```

```
Original Message: Hello, World!
Shift Value: 3
Encrypted Message: Khoor, Zruog!
Decrypted Message: Hello, World!
```

Task4–SecurityReviewof

WebServicePrompt:

Generate a simple Python Flask web service with a login endpoint that accepts username and password from a POST request. Do not include advanced security measures. After generating the code: 1. Identify at least three security vulnerabilities in the code. 2. Explain why each vulnerability is dangerous. 3. Suggest secure coding practices to fix them. 4. Provide a fully improved and secure version of the code. 5. Compare the insecure and secure versions clearly.

Code:

Simple Insecure Flask Web Service with Login Endpoint

This section provides a basic Flask web application that exposes a `/login` endpoint. It's designed to be insecure for demonstration purposes, allowing us to identify common vulnerabilities.

```
# Install Flask if you haven't already
!pip install Flask
```

```
Requirement already satisfied: Flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from Flask) (8.3.1)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.7)
```

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# A very insecure way to store users (for demonstration of vulnerabilities)
USERS = {
    "admin": "password123",
    "user": "secret"
}

@app.route('/login', methods=['POST'])
def login():
    username = request.form.get('username')
    password = request.form.get('password')

    if not username or not password:
        return jsonify({"message": "Username and password are required"}), 400
```

```
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from limits>=3.13->Flask-Limiter) (26.0)
Requirement already satisfied: wrapt<3,>=1.10 in /usr/local/lib/python3.12/dist-packages (from deprecated>=1.2->limits>=3.13->Flask-Limiter) (2.1.2)
Downloading Flask_Bcrypt-1.0.1-py3-none-any.whl (6.0 kB)
Downloading flask_session-0.8.0-py3-none-any.whl (24 kB)
Downloading flask_limiter-4.1.1-py3-none-any.whl (30 kB)
Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl (278 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 278.2/278.2 kB 19.2 MB/s eta 0:00:00
Downloading limits-5.8.0-py3-none-any.whl (60 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 61.0/61.0 kB 6.1 MB/s eta 0:00:00
Downloading msgspec-0.21.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (224 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 224.8/224.8 kB 19.2 MB/s eta 0:00:00
Downloading ordered_set-4.1.0-py3-none-any.whl (7.6 kB)
Downloading cachelib-0.13.0-py3-none-any.whl (20 kB)
Downloading deprecated-1.3.1-py2.py3-none-any.whl (11 kB)
Installing collected packages: ordered-set, msgspec, deprecated, cachelib, bcrypt, limits, Flask-Session, Flask-Limiter, Flask-Bcrypt
Successfully installed Flask-Bcrypt-1.0.1 Flask-Limiter-4.1.1 Flask-Session-0.8.0 bcrypt-5.0.0 cachelib-0.13.0 deprecated-1.3.1 limits-5.8.0 msgspec-0.21.0 ordered-set-4.1.0
Collecting redis
  Downloading redis-7.4.0-py3-none-any.whl.metadata (12 kB)
Downloading redis-7.4.0-py3-none-any.whl (409 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 409.8/409.8 kB 27.2 MB/s eta 0:00:00
Installing collected packages: redis
Successfully installed redis-7.4.0
```

```
from flask import Flask, request, jsonify, session
from flask_bcrypt import Bcrypt
from flask_session import Session # For server-side sessions
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
import secrets
import os

app = Flask(__name__)

# --- Security Configurations ---

# 1. Secret Key for Session Management
# Generate a strong secret key. In production, load from environment variable.
app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY', secrets.token_hex(32))

# 2. Session Configuration
app.config['SESSION_TYPE'] = 'filesystem' # Or 'redis', 'sqlalchemy', etc.
```

```
app.config['SESSION_PERMANENT'] = False
app.config['SESSION_USE_SIGNER'] = True # Signs the session cookie to prevent tampering
app.config['SESSION_COOKIE_SECURE'] = True # Only send cookies over HTTPS
app.config['SESSION_COOKIE_HTTPONLY'] = True # Prevent client-side JS access
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax' # Protection against CSRF
Session(app)

# 3. Bcrypt for Password Hashing
bcrypt = Bcrypt(app)

# 4. Rate Limiting for Login Attempts
# Limit to 5 attempts per minute per IP address
limiter = Limiter(
    get_remote_address,
    app=app,
    default_limits=["5 per minute"],
    storage_uri="memory://", # In production, use a persistent storage like redis or memcached
)

# --- User Management (Secure Version) ---
# In a real app, this would be a database and NOT in-memory
# Passwords are now hashed
SECURE_USERS = {
    "admin": bcrypt.generate_password_hash("securepassword123").decode('utf-8'),
    "user": bcrypt.generate_password_hash("supersecretpass").decode('utf-8')
}

# --- Routes ---

@app.route('/register', methods=['POST'])
def register_secure():
    username = request.form.get('username')
    password = request.form.get('password')

    if not username or not password:
        return jsonify({"message": "Username and password are required"}), 400

    if username in SECURE_USERS:
        return jsonify({"message": "Username already exists"}), 409 # Conflict
```

```
@app.route('/login', methods=['POST'])
@limiter.limit("5 per minute") # Apply rate limit to login endpoint
def login_secure():
    username = request.form.get('username')
    password = request.form.get('password')

    if not username or not password:
        return jsonify({"message": "Username and password are required"}), 400

    if username not in SECURE_USERS:
        return jsonify({"message": "Invalid credentials"}), 401

    # Verify the password using bcrypt
    if bcrypt.check_password_hash(SECURE_USERS[username], password):
        session['username'] = username # Establish user session
        return jsonify({"message": "Login successful", "user": username}), 200
    else:
        return jsonify({"message": "Invalid credentials"}), 401

@app.route('/protected')
def protected_route():
    if 'username' in session:
        return jsonify({"message": f"Welcome, {session['username']}! This is a protected area.", "user": session['username']}), 200
    return jsonify({"message": "Unauthorized: Please log in."}), 401

@app.route('/logout')
def logout():
    session.pop('username', None) # Remove user from session
    return jsonify({"message": "Logged out successfully"}), 200

@app.route('/')
def index_secure():
    return "Welcome to the SECURE Flask app! Try POSTing to /register or /login, or accessing /protected."


print("To run the SECURE Flask app, execute the following line in a new cell:")
print("from werkzeug.serving import run_simple; run_simple('127.0.0.1', 5000, app, use_debugger=True, use_reloader=False)")
```

```
            return jsonify({"message": "Username and password are required"}), 400

        if username in USERS and USERS[username] == password:
            return jsonify({"message": "Login successful", "user": username}), 200
        else:
            return jsonify({"message": "Invalid credentials"}), 401

    @app.route('/')
    def index():
        return "Welcome to the insecure Flask app! Try POSTing to /login"

    # To run this in Colab, you'll need to expose it.
    # Using ngrok is a common way, but for local testing within Colab,
    # you might need to adjust based on Colab's network setup or just use Werkzeug's dev server.
    # For a simple run in Colab, you can use:
    # from werkzeug.serving import run_simple
    # run_simple('127.0.0.1', 5000, app, use_debugger=True, use_reloader=False)

    # In a typical development environment, you'd run:
    # if __name__ == '__main__':
    #     app.run(debug=True, port=5000)

    # For demonstration in Colab, we'll start a simple server that's accessible
    # You might need to expose it through a service like ngrok for external access
    # For testing locally within Colab, this will work:
    # To stop it, you'll need to interrupt the kernel.
    # If you run this cell, it will block execution until stopped or the app crashes.

    print("To run the Flask app, execute the following line in a new cell:")
    print("from werkzeug.serving import run_simple; run_simple('127.0.0.1', 5000, app, use_debugger=True, use_reloader=False)")
```

```
To run the Flask app, execute the following line in a new cell:
from werkzeug.serving import run_simple; run_simple('127.0.0.1', 5000, app, use_debugger=True, use_reloader=False)
```

```
To run the SECURE Flask app, execute the following line in a new cell:
from werkzeug.serving import run_simple; run_simple('127.0.0.1', 5000, app, use_debugger=True, use_reloader=False)
```

Comparison: Insecure vs. Secure Flask Web Service

Let's highlight the key differences and improvements between the two versions:

1. Password Storage

- **Insecure Version:**
  - `USERS = {"admin": "password123", "user": "secret"}`
  - Passwords stored in plain text.
- **Secure Version:**
  - `SECURE_USERS = {"admin": bcrypt.generate_password_hash("securepassword123").decode('utf-8'), ...}`
  - Uses `Flask-Bcrypt` to **hash passwords** with a strong, one-way algorithm (bcrypt), making them virtually impossible to reverse engineer. `generate_password_hash` automatically handles salting.
  - A `register` endpoint was added to demonstrate how new users would securely store their passwords.

2. Session Management

- **Insecure Version:**
  - No session management. Login merely returned a success message, but no persistent authentication state was established.
- **Secure Version:**
  - Uses `Flask-Session` for **server-side session management** (`app.config['SESSION_TYPE'] = 'filesystem'`).
  - **Secure session cookies** configured (`SESSION_COOKIE_SECURE` , `SESSION_COOKIE_HTTPONLY` , `SESSION_COOKIE_SAMESITE`) to prevent tampering and various attacks.
  - `session['username'] = username` is used to store authentication status after a successful login.
  - **Secret Key** configured (`app.config['SECRET_KEY']`) to cryptographically sign session cookies, preventing modification by clients.

- **Secret Key** configured (`app.config['SECRET_KEY']`) to cryptographically sign session cookies, preventing modification by clients.
- Added a `/protected` route that requires an active session and a `/logout` route to clear the session.

3. Rate Limiting

- **Insecure Version:**
  - No rate limiting on the login endpoint.
- **Secure Version:**
  - Uses `Flask-Limiter` to **limit login attempts** (`@limiter.limit("5 per minute")`). This significantly mitigates brute-force attacks by slowing down attackers and potentially leading to account lockouts (though a full lockout mechanism would require database integration).

4. User Data Storage

- **Insecure Version:** In-memory dictionary `USERS` .
- **Secure Version:** Still uses an in-memory dictionary `SECURE_USERS` for demonstration simplicity, but the comments explicitly state: "In a real app, this would be a database and NOT in-memory." A real-world application would use a proper database (e.g., PostgreSQL, MySQL) with an ORM (like SQLAlchemy) for user data storage and retrieval.

5. Error Handling for Authentication

- **Insecure Version:** Provides generic "Invalid credentials" for both incorrect password and non-existent username.
- **Secure Version:** The secure version also returns "Invalid credentials" for both incorrect password and non-existent username, which is a good security practice to prevent username enumeration attacks (where an attacker could guess valid usernames by distinguishing between 'user not found' and 'incorrect password' messages).

This secure version provides a much more robust foundation for a web application, protecting against common and critical security threats.

Task5–ExceptionHandlingImprovementPrompt:

Generate Python calculator

Code:

Simple Python Calculator

This calculator program defines functions for basic arithmetic operations: addition, subtraction, multiplication, and division. It then demonstrates how to use these functions with example inputs.

```python
def add(x, y):
    """Adds two numbers."""
    return x + y

def subtract(x, y):
    """Subtracts two numbers."""
    return x - y

def multiply(x, y):
    """Multiplies two numbers."""
    return x * y

def divide(x, y):
    """Divides two numbers, handles division by zero."""
    if y == 0:
        return "Error! Division by zero."
    return x / y
```

Demonstration of the Calculator

```python
# Example usage of the calculator functions

num1 = 10
num2 = 5

print(f"{num1} + {num2} = {add(num1, num2)}")
print(f"{num1} - {num2} = {subtract(num1, num2)}")
print(f"{num1} * {num2} = {multiply(num1, num2)}")
print(f"{num1} / {num2} = {divide(num1, num2)}")

# Test division by zero
num3 = 0
print(f"{num1} / {num3} = {divide(num1, num3)}")
```

```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.0
10 / 0 = Error! Division by zero.
```